Rain-Robust Pothole Detection

How much does rain hurt a pothole detector, and what helps?

CSE36501 Deep Learning — Project Proposal

[Name] ([Student ID])

November 6, 2026

UNIST, Fall 2026

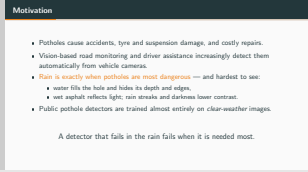
Rain-Robust Pothole Detection
How much does rain hurt a pothole detector, and what helps?
CSE36501 Deep Learning — Project Proposal
[Name] ([Student ID])
November 6, 2026
UNIST, Fall 2026

Hello, I am [Name]. My project is about rain-robust pothole detection. In short, I want to measure how much a pothole detector that was trained on clear-weather images breaks down when it rains, and then test which fix helps the most. In this proposal I will cover the problem definition and the datasets; models and experiments will come in the final presentation.

Motivation

- Potholes cause accidents, tyre and suspension damage, and costly repairs.
- Vision-based road monitoring and driver assistance increasingly detect them automatically from vehicle cameras.
- Rain is exactly when potholes are most dangerous — and hardest to see:
  - water fills the hole and hides its depth and edges,
  - wet asphalt reflects light; rain streaks and darkness lower contrast.
- Public pothole detectors are trained almost entirely on *clear-weather* images.

A detector that fails in the rain fails when it is needed most.



Potholes are not just an annoyance: they damage vehicles and cause accidents, and many cities and car makers now try to detect them automatically from cameras. The problem is that rain is precisely the condition where potholes are most dangerous, because water fills them and hides their depth, and wet asphalt reflects light. At the same time, almost all public training data is captured in good weather. So there is a mismatch between where these models are trained and where they matter most.

Problem statement

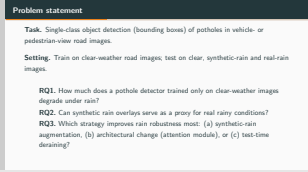
Task. Single-class object detection (bounding boxes) of potholes in vehicle- or pedestrian-view road images.

Setting. Train on clear-weather road images; test on clear, synthetic-rain and real-rain images.

RQ1. How much does a pothole detector trained only on clear-weather images degrade under rain?

RQ2. Can synthetic rain overlays serve as a proxy for real rainy conditions?

RQ3. Which strategy improves rain robustness most: (a) synthetic-rain augmentation, (b) architectural change (attention module), or (c) test-time deraining?



Formally, the task is single-class pothole detection with bounding boxes. I train on clear-weather images and test under three conditions: clear, synthetic rain, and real rain. This gives three research questions. First, how large is the drop under rain? Second, does synthetic rain behave like real rain, so that we could benchmark and train with cheap synthetic data? And third, among three common fixes, augmentation, an attention module, and a deraining pre-processor, which one actually helps most?

Datasets

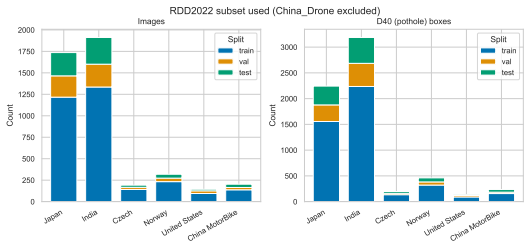
	RDD2022 (Arya et al., 2024)	MWPD (Parvin et al., 2025)
Role	clear-weather train / val / test	external test: real normal vs. rain
Source	vehicle-mounted cameras & phones, 6 countries	mixed web, news and phone images
Size	47,420 images, 4 damage types; we keep **D40 = pothole** (3,610 positive images, 6,458 boxes)	1,357 images, 640×640
Labels, license	Pascal VOC XML; CC BY 4.0	YOLO txt, 1 class; CC BY 4.0
Conditions	clear daylight	normal / rainy; day / twilight / night

- **Why these:** RDD2022 is the largest public multi-country road-damage set; MWPD is, to our knowledge, the only public pothole set with real rain.
- China_Drone excluded (top-down drone view $\neq$ vehicle view).
- MWPD ships without weather labels $\Rightarrow$ every image was tagged normal/rainy (pre-tag + manual review).

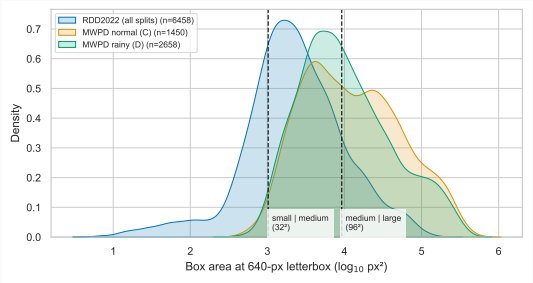


I use two datasets. RDD2022 is the large multi-country road damage dataset from the CRDDC challenge, with about forty-seven thousand images. It has several damage classes; I keep only D40, which is the pothole class. I drop the China drone subset because it is shot from above and does not match a car's view. For real rain I use MWPD, the Multi-Weather Pothole Detection dataset, which contains normal and rainy images. One important finding: although MWPD describes rainy conditions, the files carry no weather label, so I tagged every image as normal or rainy myself.

Dataset analysis

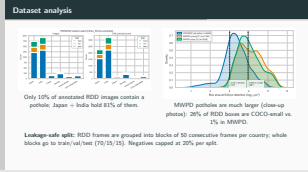


Only 10% of annotated RDD images contain a pothole; Japan + India hold 81% of them.



MWPD potholes are much larger (close-up photos): 26% of RDD boxes are COCO-small vs. 1% in MWPD.

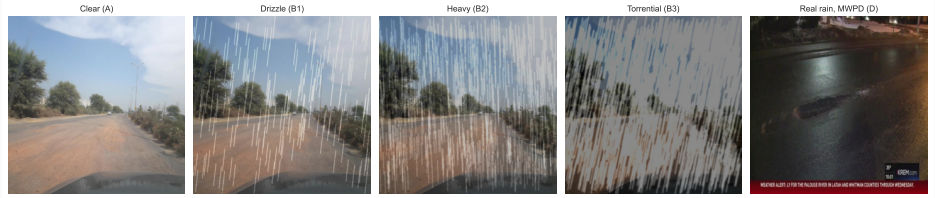
Leakage-safe split: RDD frames are grouped into blocks of 50 consecutive frames per country; whole blocks go to train/val/test (70/15/15). Negatives capped at 20% per split.



Two things stand out in the data. On the left: potholes are rare in RDD; only about one in ten annotated images contains one, and most come from Japan and India. On the right: object size differs a lot. RDD potholes are small and far away, as seen from a dashcam, while MWPD potholes are mostly large close-ups. So part of any drop from RDD to MWPD will be a size and viewpoint effect, not weather, and I will report size-wise AP to separate them. Finally, because RDD contains video-like consecutive frames, I split by blocks of fifty frames so near-identical images never end up in both train and test.

Test-set design

ID	Composition	Images	Purpose
A	RDD2022 own test split (clear)	708	baseline
B1/B2/B3	*same images* as A + synthetic drizzle / heavy / torrential	708 each	pure rain effect
C	MWPD normal weather	681	domain gap
D	MWPD rainy	676	real rain effect



A→B: synthetic rain only A→C: domain (country, camera, size) C→D: real rain, same source

(A→B) vs. (C→D): is synthetic rain a valid proxy? (RQ2)

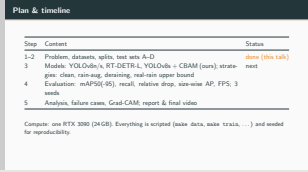


This is the core of the design. Set A is my clean RDD test split. B1 to B3 are the very same images with synthetic rain of increasing strength, so any change from A to B is caused by rain alone. C and D come from MWPD: normal and rainy. A to C measures the domain gap, and C to D measures the effect of real rain within the same source. Comparing the A-to-B drop with the C-to-D drop answers whether synthetic rain is a fair stand-in for real rain. Note that synthetic rain only adds streaks, darkening and blur; it cannot reproduce water-filled potholes or reflections, which is exactly what we may see in D.

Plan & timeline

Step	Content	Status
1–2	Problem, datasets, splits, test sets A–D	done (this talk)
3	Models: YOLOv8n/s, RT-DETR-L, YOLOv8s + CBAM (ours); strategies: clean, rain-aug, deraining, real-rain upper bound	next
4	Evaluation: mAP50(-95), recall, relative drop, size-wise AP, FPS; 3 seeds	
5	Analysis, failure cases, Grad-CAM; report & final video	

Compute: one RTX 3090 (24 GB). Everything is scripted (`make data`, `make train`, ...) and seeded for reproducibility.



Steps one and two, the problem and the data, are finished and are what I presented today. Next I will train four detectors, two YOLOv8 sizes, the transformer-based RT-DETR, and my own YOLOv8 variant with CBAM attention, under clean training, synthetic-rain augmentation, test-time deraining, and real-rain training as an upper bound. I will evaluate them on all six test sets with three seeds each, and finish with failure analysis and Grad-CAM. Thank you.